Model Parallelism in LLM Training

Spring 2026

Lecturer: Yuedong (Steven) Xu

Fudan University

ydxu@fudan.edu.cn

Disclaimer

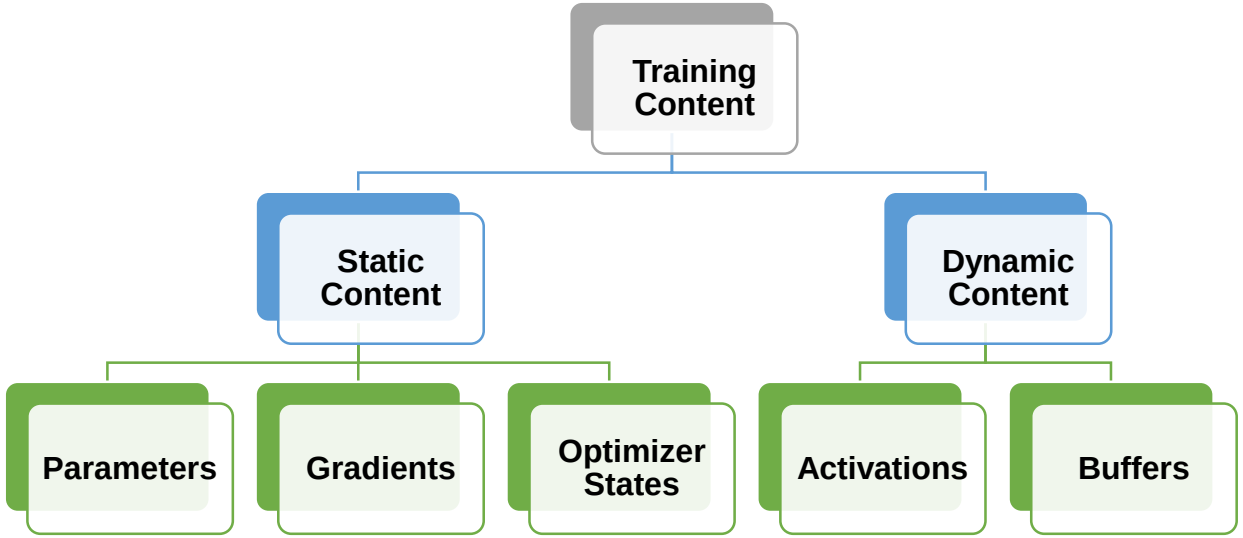
Machine learning systems is a broad and rapidly evolving field. The course material has been developed using a broad spectrum of resources, including research papers, lecture slides, blogposts, research talks, tutorial videos, and other materials shared by the research community.

Distributed LLM Training: Outline

- Data Parallelism
  - Parameter-Server
  - All-Reduce
  - Memory Optimization
- Model Parallelism
  - **Pipeline Parallelism**
  - Tensor Parallelism
  - Sequence Parallelism
- Mixture of Experts

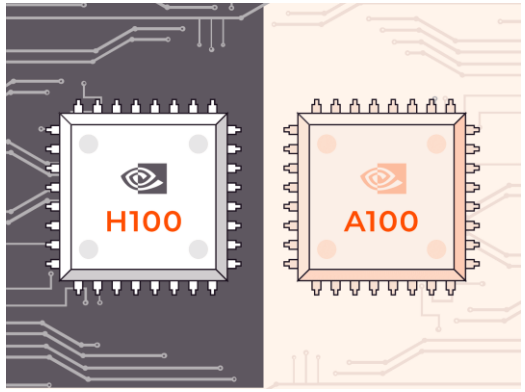
Pipeline Training

- Retrospect: GPU HBM Content During Training

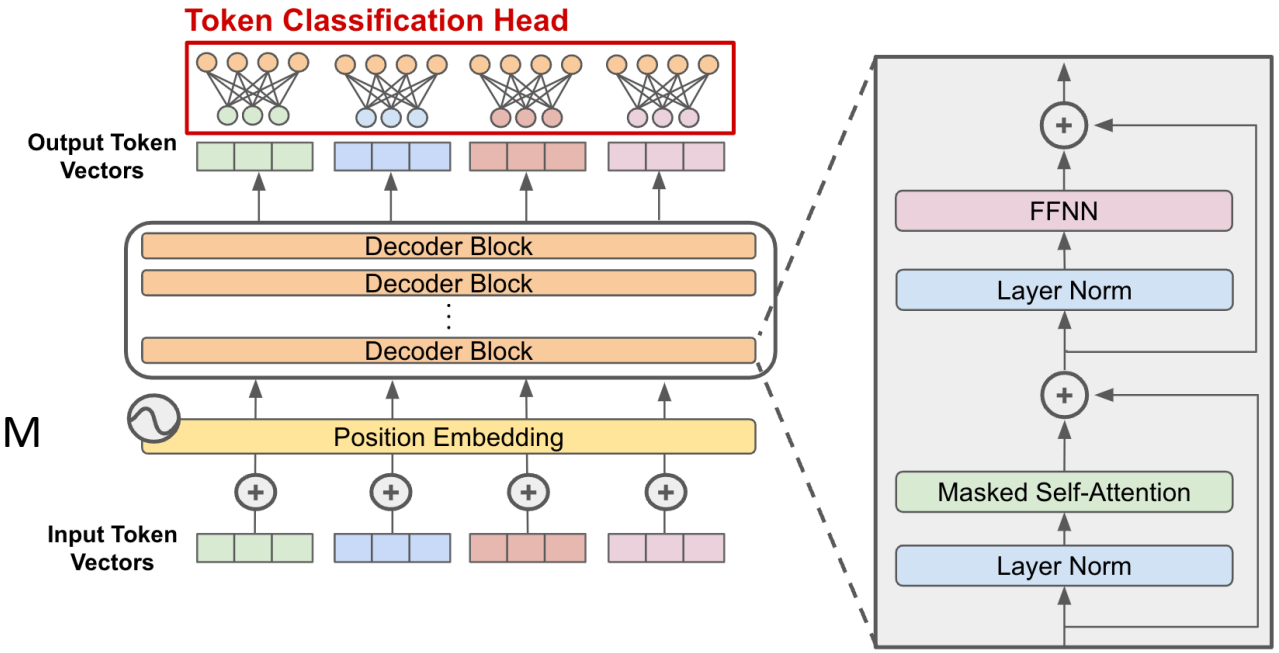


- An example of GPT-3 175B
  - **Optimizer States**
    - 32-bit Parameter (700 GB)
    - Adam Moment (700 GB)
    - Adam Variance (700 GB)
  - **16-bit Parameter (350 GB)**
  - **16-bit Gradient (350 GB)**
  - **Activations (depending on batch size)**
  - Buffer and Fragmentation

Pipeline Training



Insufficient HBM

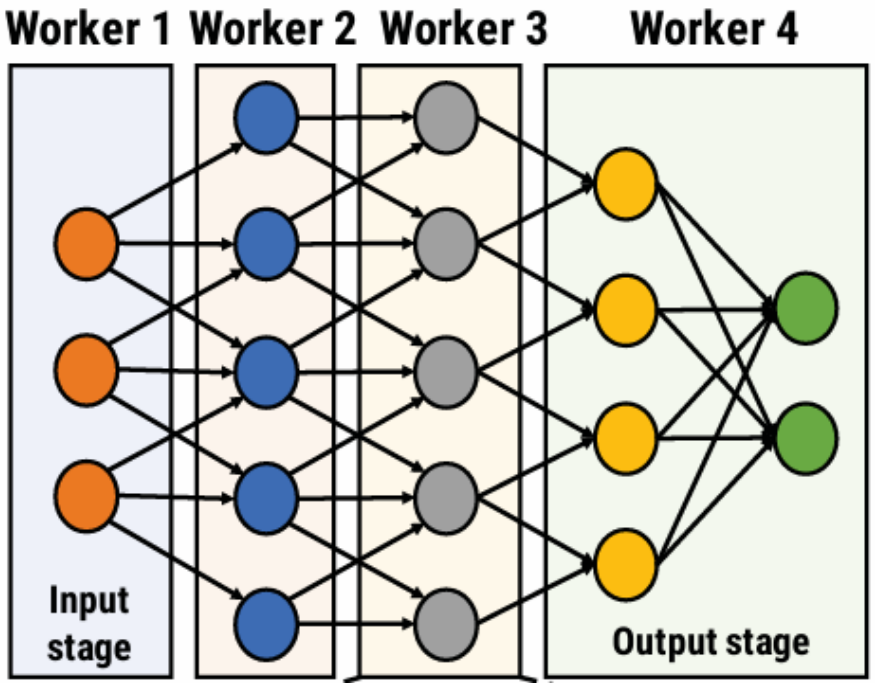


When LLM model is too LARGE!

Pipeline Training

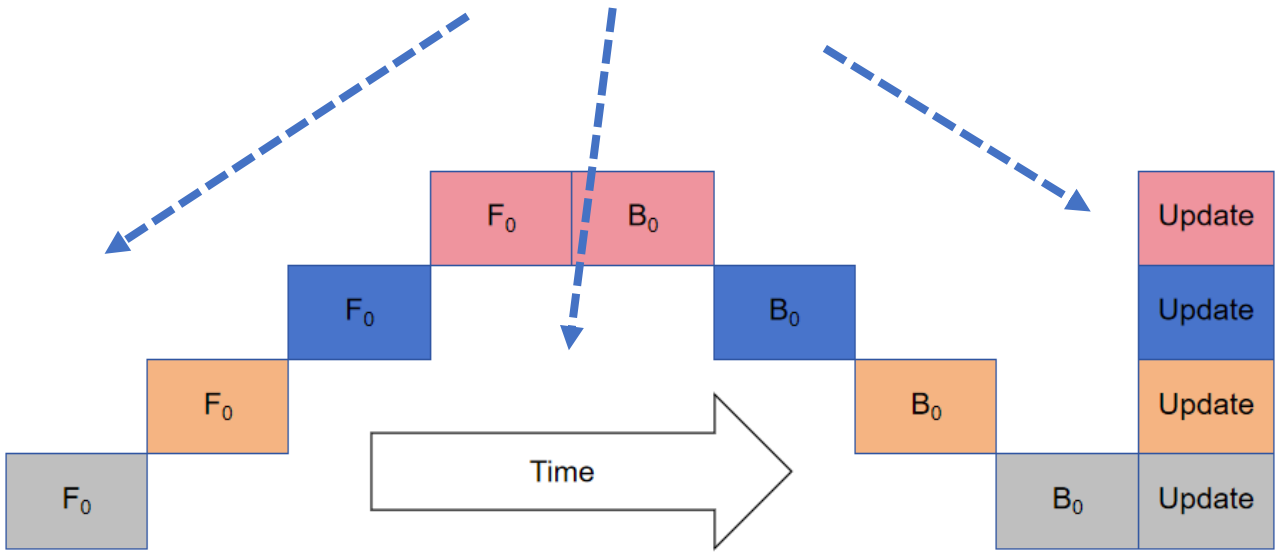
- Letting data parallelism be SUFFICIANTLY large
  - Linearly increasing network latency (αn)
  - Inefficient small data chunk transmission
  - Straggler effect caused by any communication pair

Pipeline Training



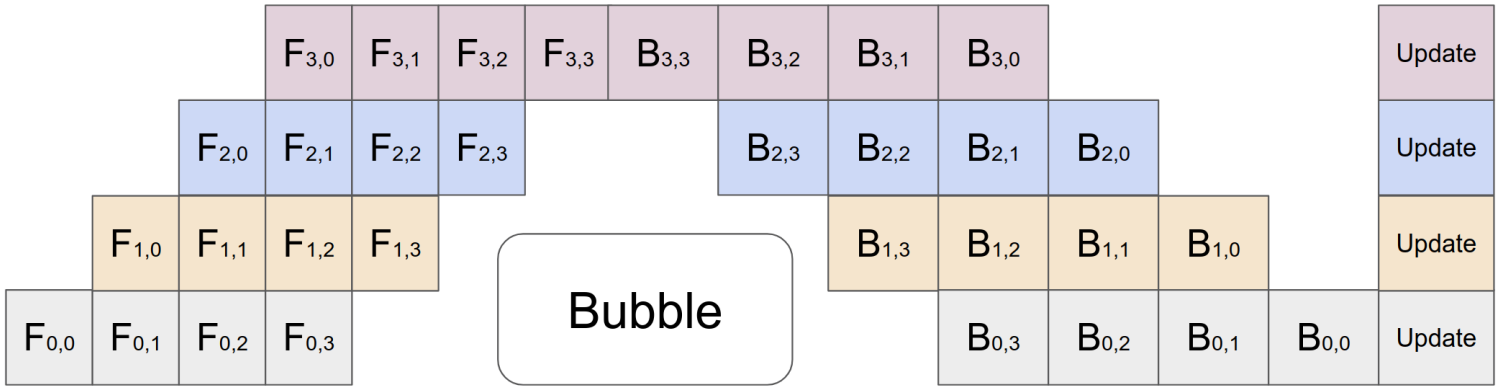
Layerwise partitioning on multiple GPUs

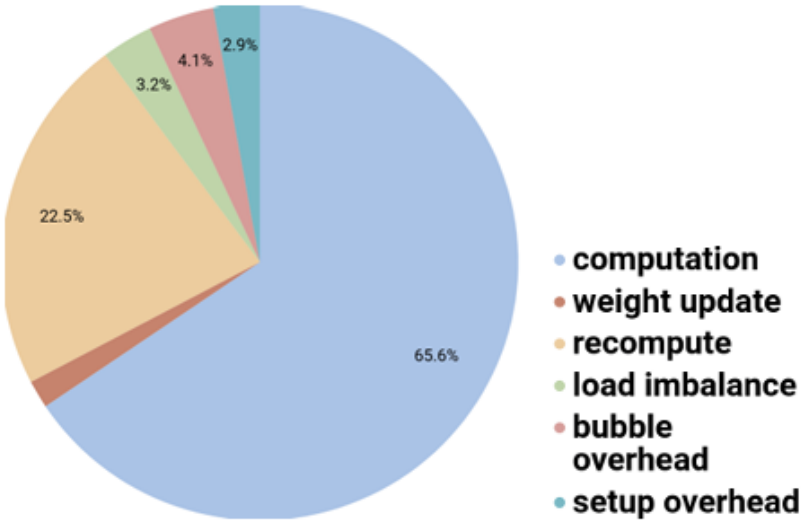
Large bubble $O(\frac{K-1}{K})$ → underutilized GPU



Naïve model parallelism
(F for forward； B for backward)

Pipeline Training



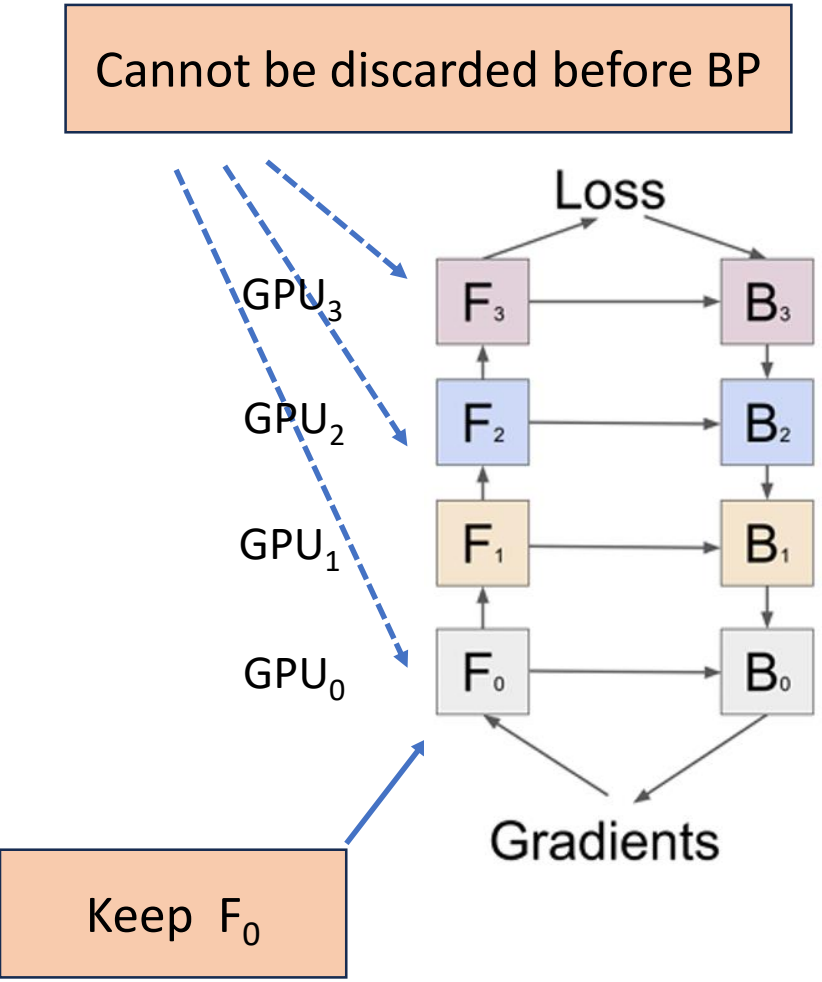


b: batch size
b_m : microbatch size
K: # pipeline stages
M: # microbatches, b = b_m *M

Bubble ratio: $O(\frac{K-1}{K+M-1})$

《GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism》

Pipeline Training

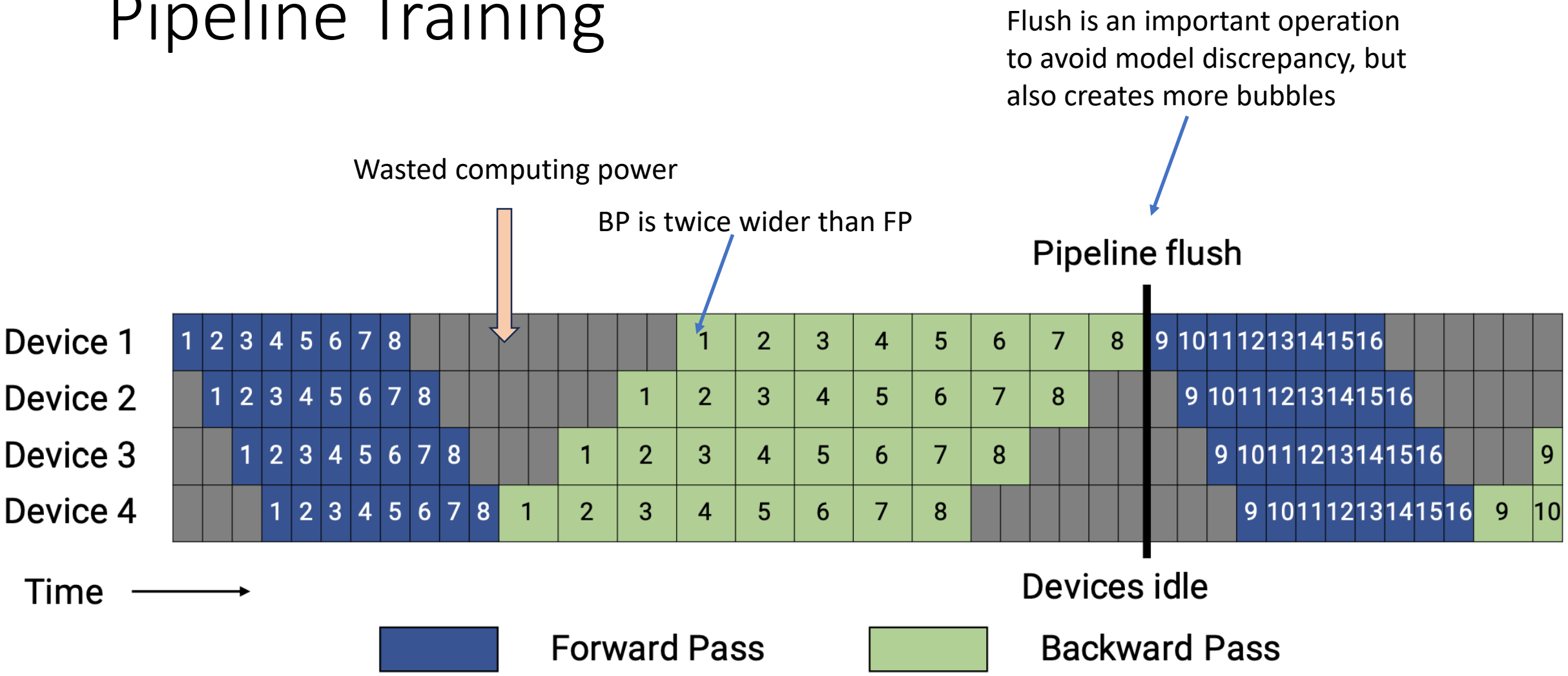


- Computation time
  - Forward time is half of backward time (why?)
- Communication load
  - Intermediate activation between two partitions
- Memory footprint
  - First stage usually needs more memory: $K * M$
  - Re-materialization (re-computation) Further reducing memory consumption

F-then-B: Forward then Backward

《GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism》

Pipeline Training



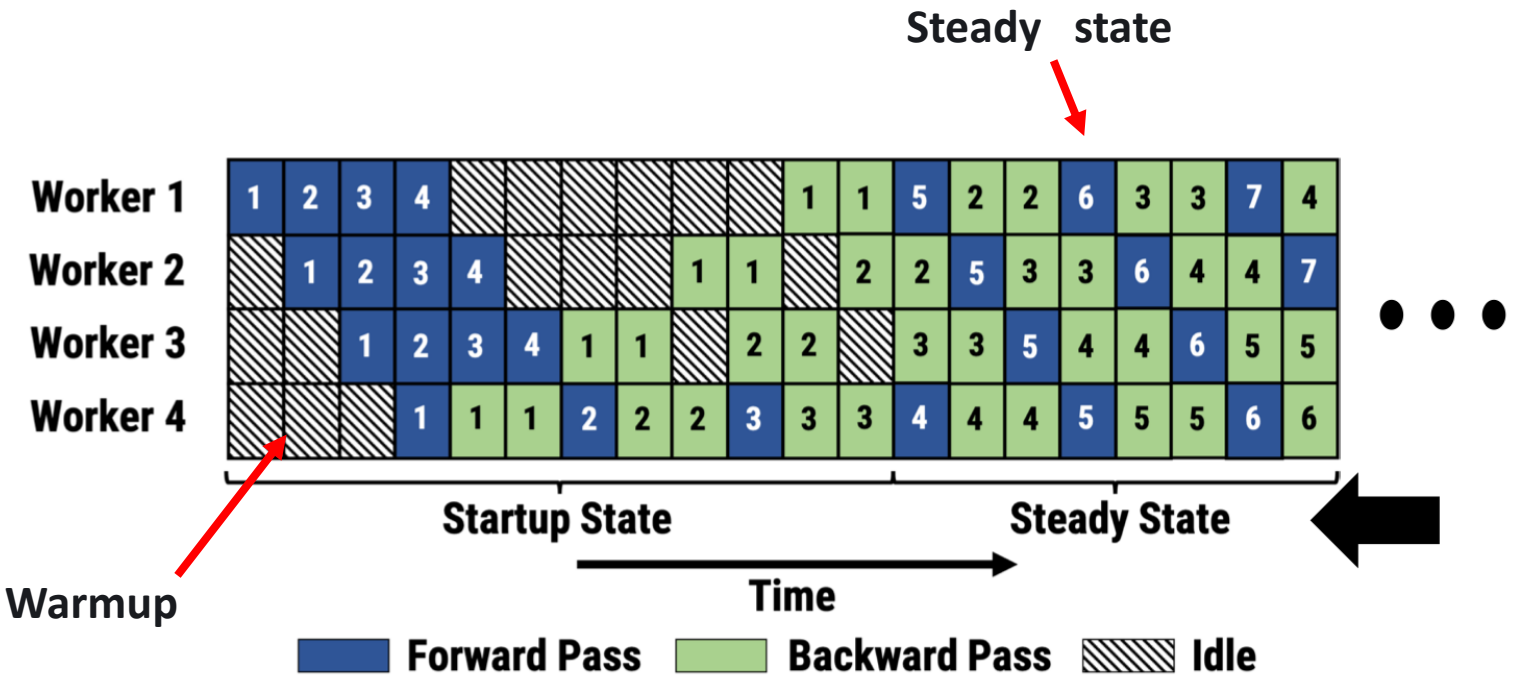
Frequent pipeline flushes lead to increased idle time

Pipeline Training

Orchestrating execution orders of forward and backward passes for microbatch so as to reduce bubbles and peak memory footprints?

- Some KEY concepts
  - Synchronous pipeline versus asynchronous pipeline
  - Interleaved versus non-interleaved

Pipeline Training

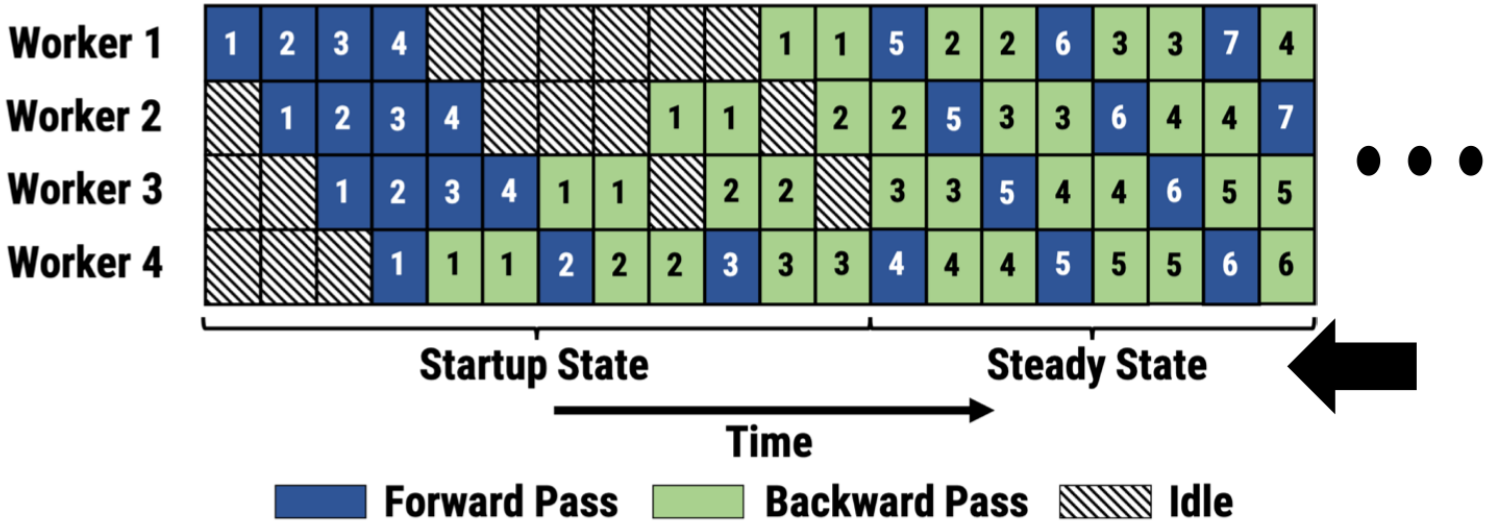


1F1B: One Forward then One Backward

- Minimizing lingering time of activations
  - Immediately executing BP after FP for the first microbatch at the output stage
- At steady state
  - Strictly alternative Forward and Backward operations
- Asynchronous weights

《PipeDream: generalized pipeline parallelism for DNN training》

Pipeline Training



1F1B: One Forward then One Backward

- Challenges
  - How to split a DNN model into different stages?

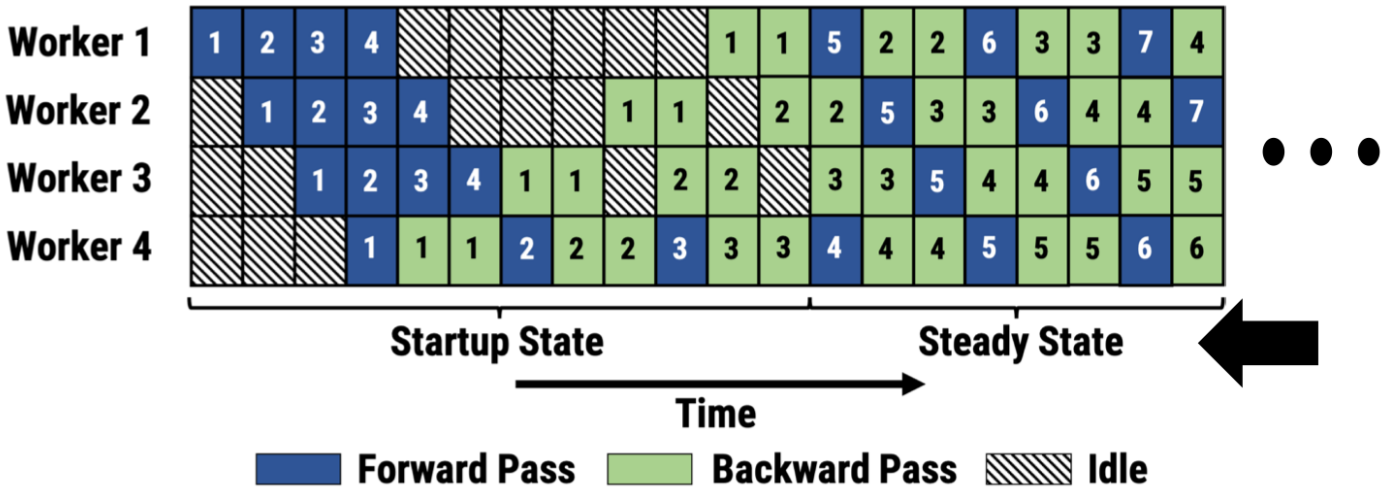
    LOAD BALANCING *across stages*

  - How to update parameters?

    After backward pass, the gradient can be used ***immediately to update the parameter***

《PipeDream: generalized pipeline parallelism for DNN training》

Pipeline Training



- Observations in stage 1
  - The forward pass of microbatch 5 is performed after the update of **microbatch 1**
  - The backward pass of microbatch 5 is performed after the updates of **microbatches 2, 3 and 4**

Forward and backward passes operate on different versions of parameters!

《PipeDream: generalized pipeline parallelism for DNN training》

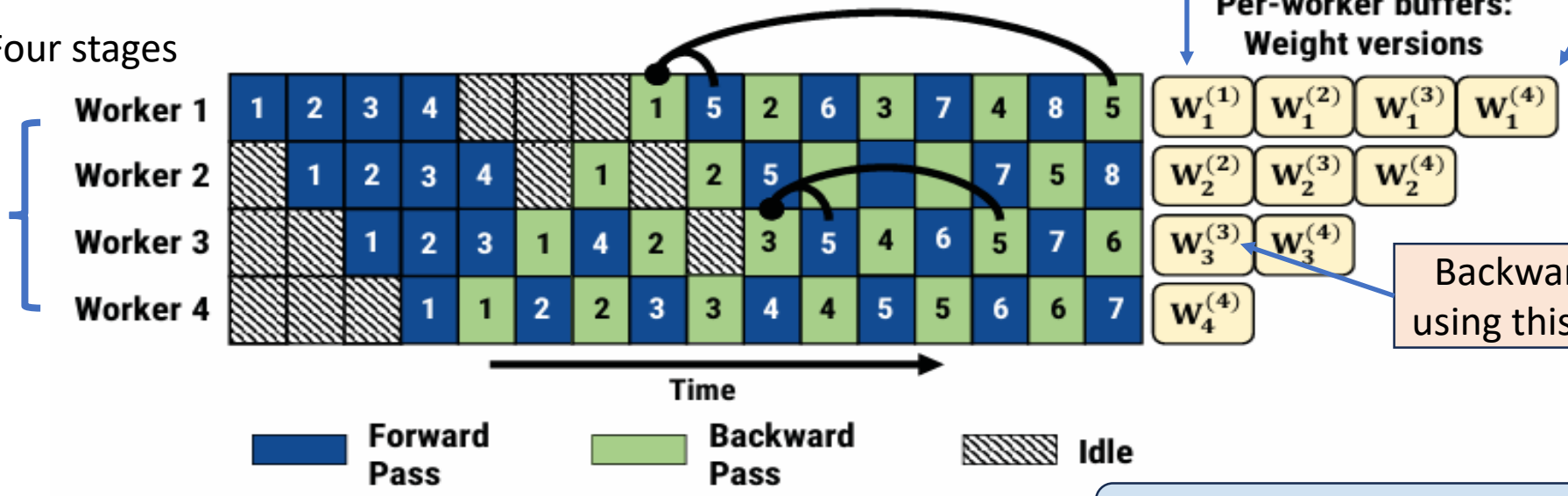
Pipeline Training

Subscript: stage ID, superscript: model version ID

Backward pass using this version

Per-worker buffers: Weight versions

Four stages



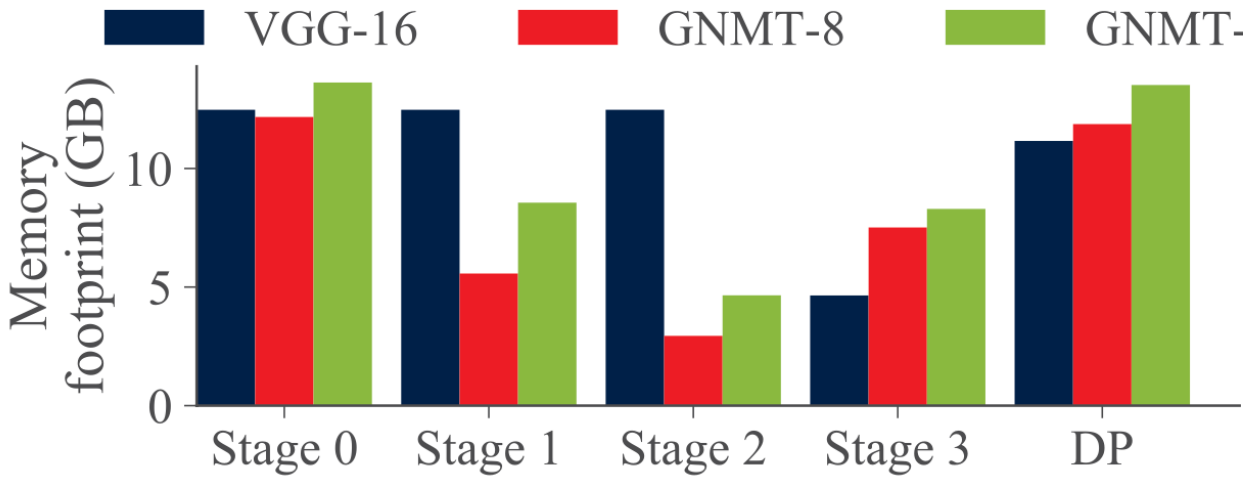
Backward pass using this version

No guarantees across stages!

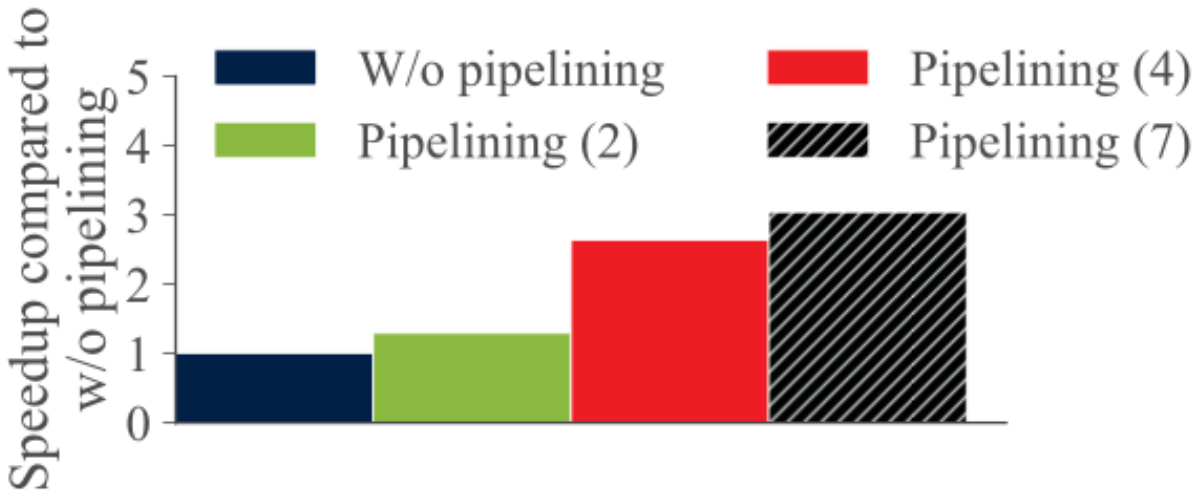
- ***Weight stashing*** of PipeDream
  - keeps multiple versions of model weights—one for each active minibatch.
  - Forward pass: each pipeline stage uses the most recent weight version available. Once the forward pass for a minibatch is completed, the corresponding weight version is stored.
  - Backward pass: the same version is reused to compute gradients and update weights, ensuring that both passes for a given minibatch use identical parameters within a stage.

Pipeline Training

- Experiments



Memory footprint of PipeDream

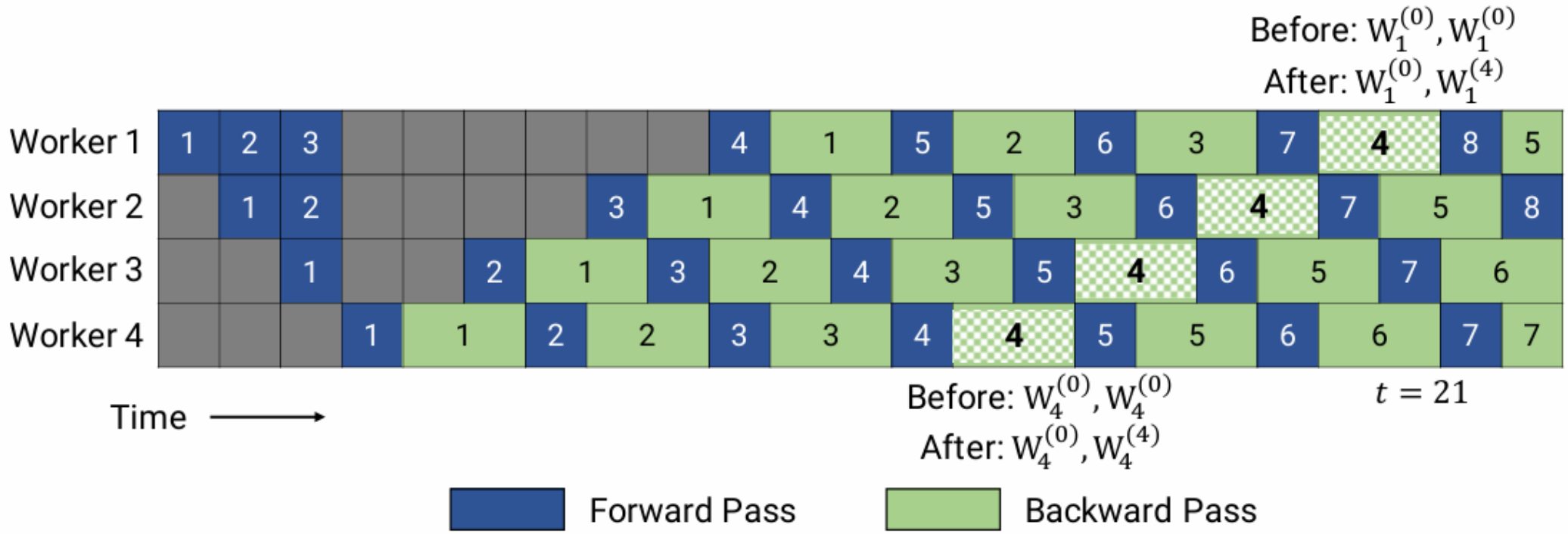


Speedup ratios on 4 V100 GPUs

PipeDream still saves memory by trading multiple versions of parameters for large activations

Subscript: stage ID, superscript: model version ID

Pipeline Training



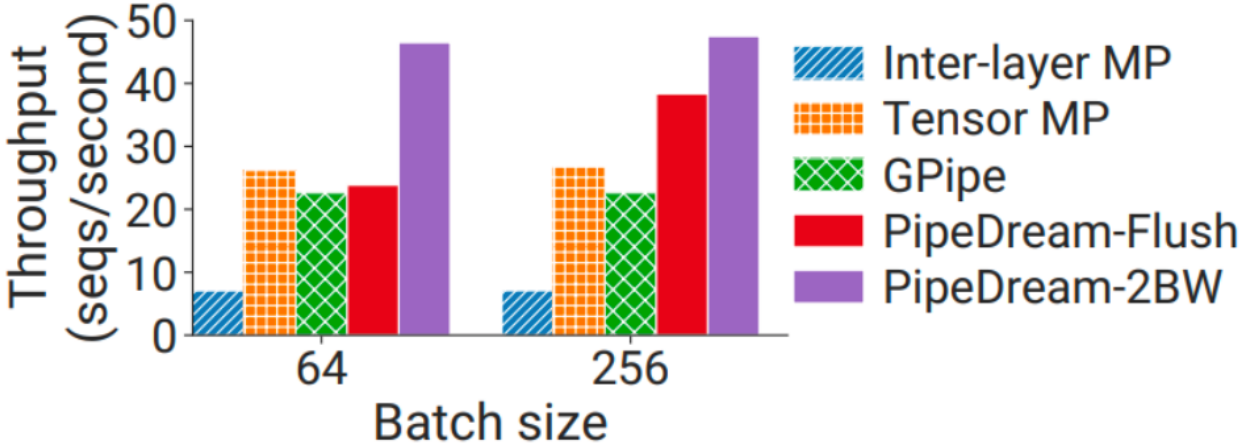
PipeDream-2BW: (1) less frequent flushing than GPipe (2) keeping no more than two model versions.

Think why? How does it relate to pipeline depth K and number of microbranches M

《Memory-Efficient Pipeline-Parallel DNN Training》

Pipeline Training

- Flushing causes more bubbles
  - Warmup + Colddown
- Flushing needs less memory



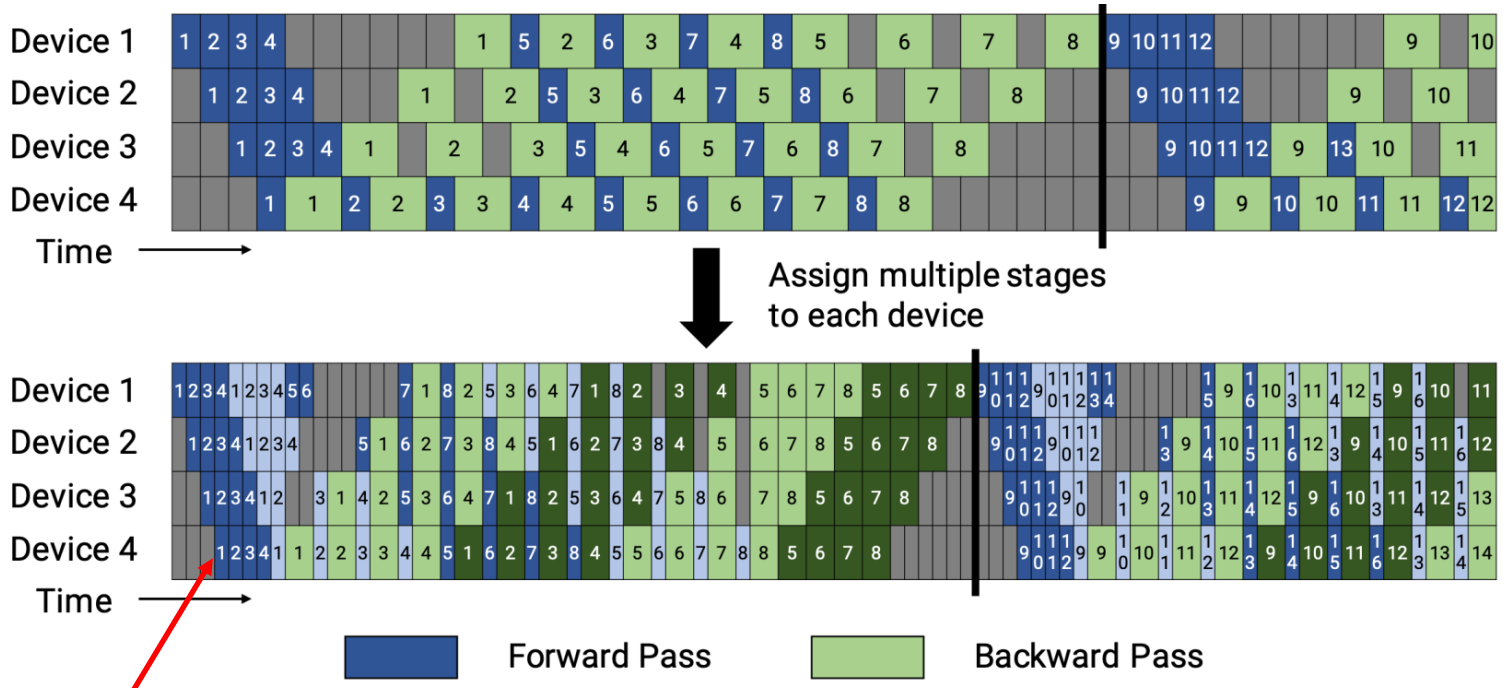
- Training BERT 2.2B with 8-way model parallelism (8 V100 GPU)
- PipeDream is much better than vannila GPipe

《Memory-Efficient Pipeline-Parallel DNN Training》

https://juejin.cn/post/7262274383287484476

Pipeline Training

- IFIB integrated in Megatron-LM later on



Interleaved 1F1B

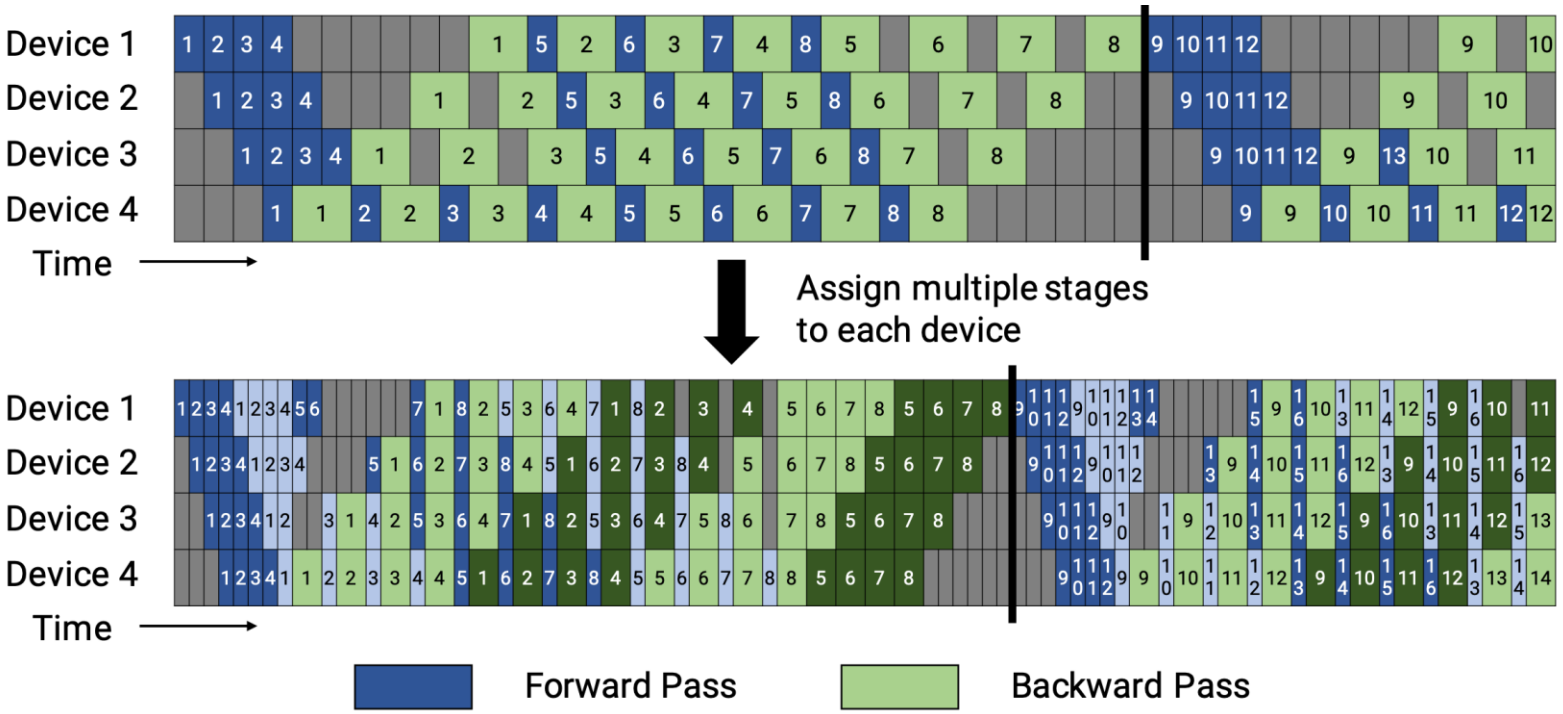
(Stages: 4, Layers: 16, each device assigned with 2 chunks. Layer 0-1, 8-9 on Device 1; Layer 2-3,10-11 on Device 2; Layer 4-5, 12-13 on Device 3 and Layer 6-7, 14-15 on Device4)

- Key ides:
  - Further dividing each microbatch into even smaller chunks
  - Microbatch and small chunks execute alternatively
- Advantages/disadvantages
  - Smaller bubble
  - Higher inter-state traffic load

《Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM》

Pipeline Training

- IFIB integrated in Megatron-LM later on



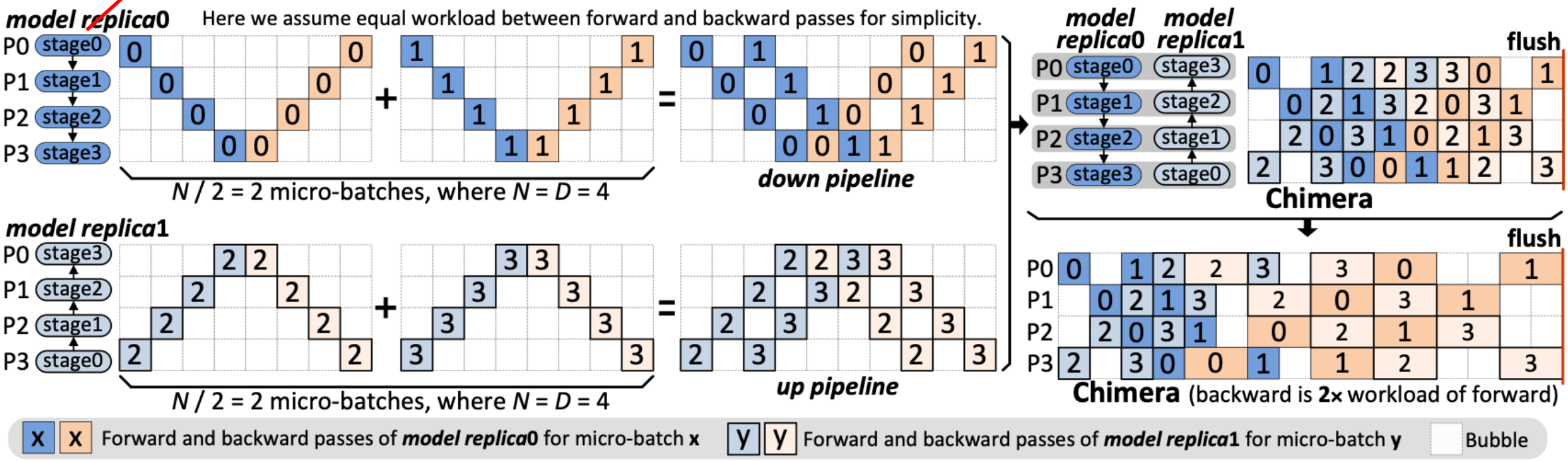
- Model placement:
  - Stages: 4, Layers: 16, each device assigned with 2 chunks
  - Layer 0-1, 8-9 on Device 1
  - Layer 2-3,10-11 on Device 2
  - Layer 4-5, 12-13 on Device 3
  - Layer 6-7, 14-15 on Device 4

Be aware of the communication scheduling!

《Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM》

Pipeline Training

Storing the first partition of model0 and the fourth partition of model1



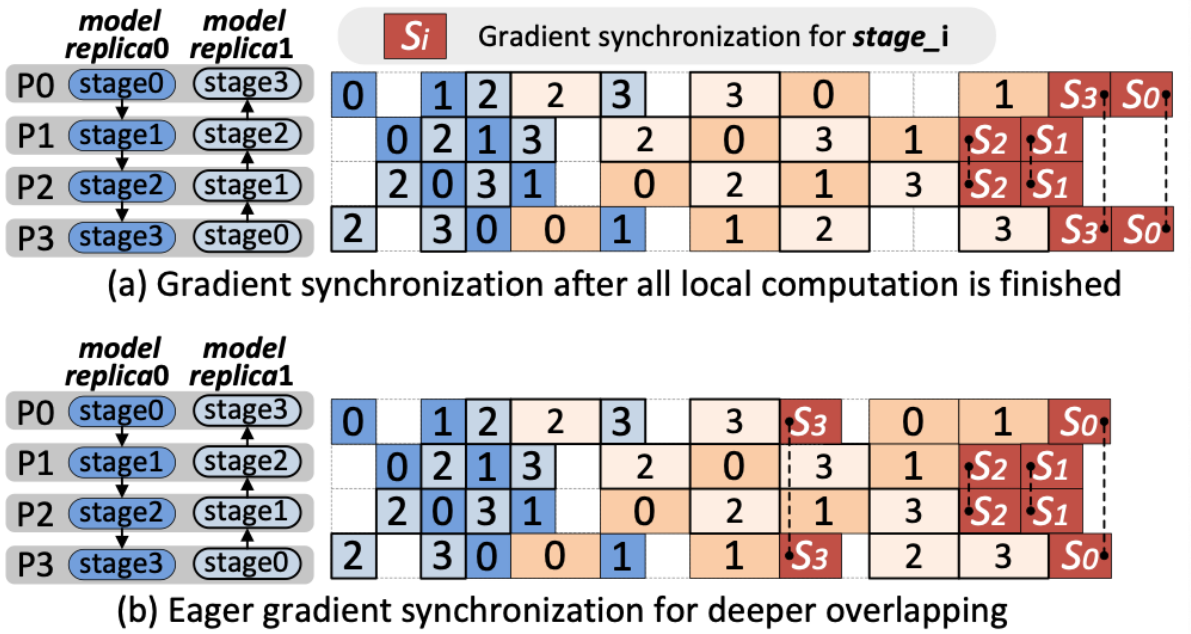
Chimera: double pipeline

- Smaller bubbles yet doubled memory occupation for model replicas

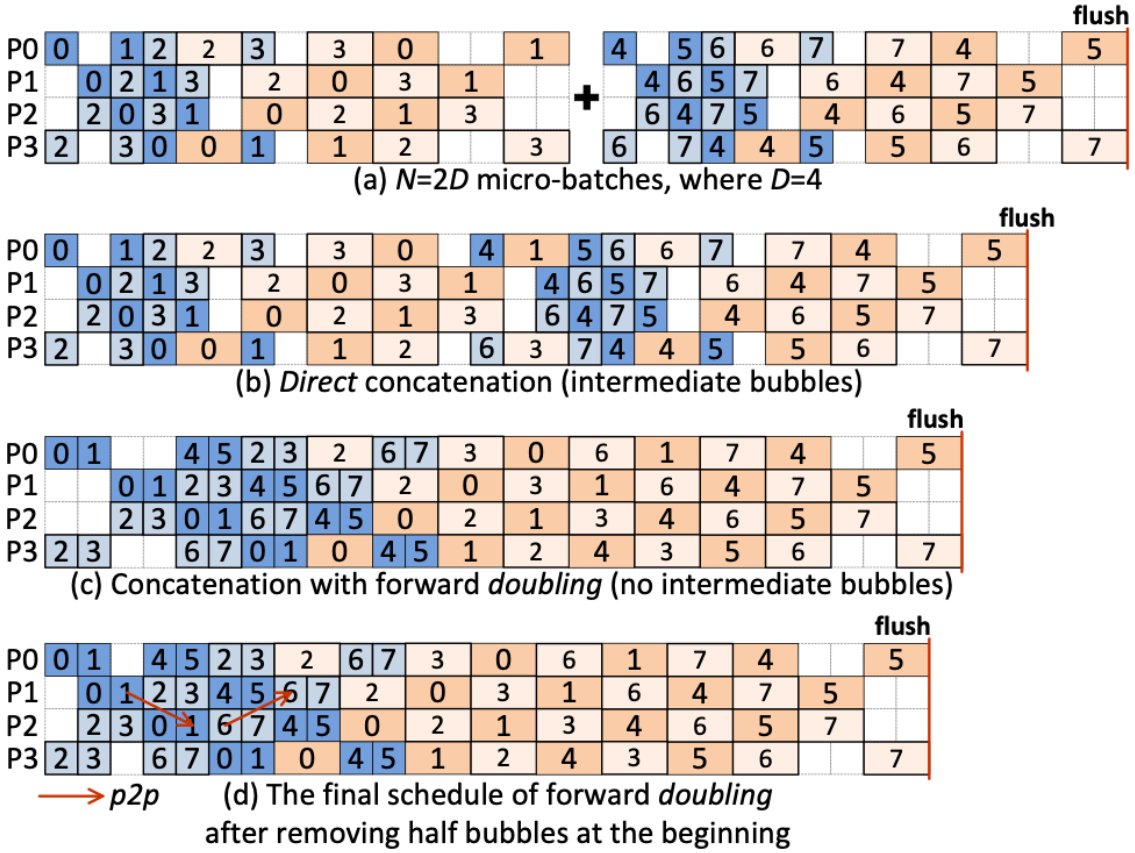
《Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines》

Pipeline Training

- Orchestrating bubble mitigation and data-parallel synchronization



Hide the gradient synchronization overhead by overlapping

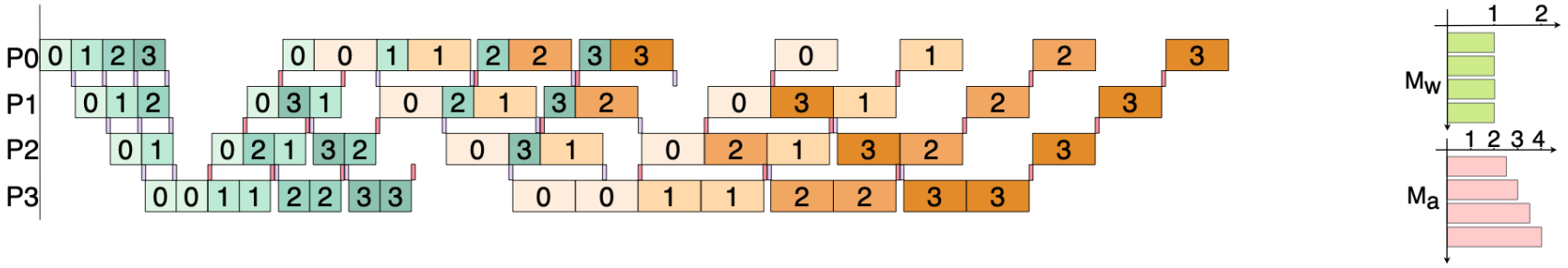


Select # replicas, # stages and Assemble blocks

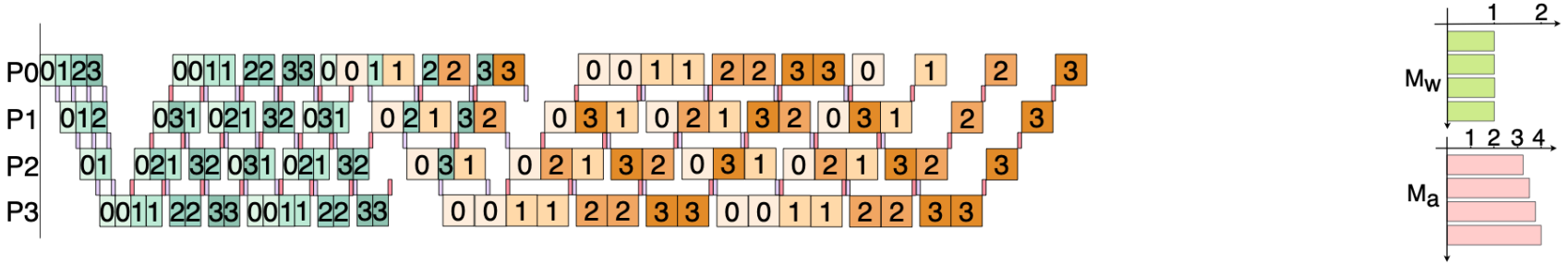
《Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines》

Pipeline Training

- Using multiple WAVEs by dividing a microbatch into multiple smaller microbatches



(d) Hanayo with one wave



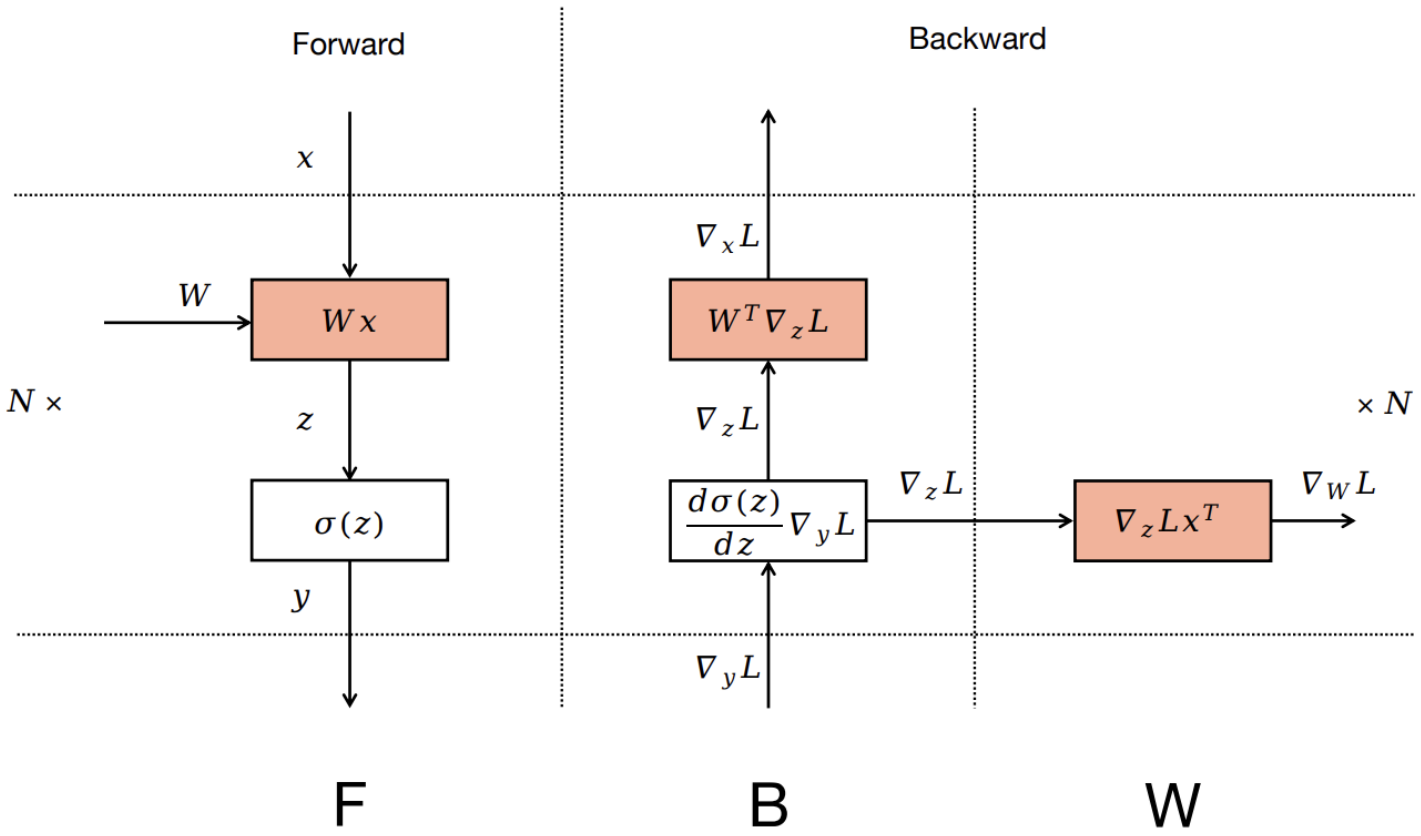
(e) Hanayo with two waves

Forward propagation marked green and Backward propagation marked orange. Back propagation illustrated twice as long as forward propagation. Using purple and pink blocks to represent P2P communication that goes downward and upward. The numbers on the blocks show which of the micro-batches is being processed.

《Hanayo: Harnessing Wave-like Pipeline Parallelism for Enhanced Large Model Training Efficiency》

Pipeline Training

- A microscopic view of BP

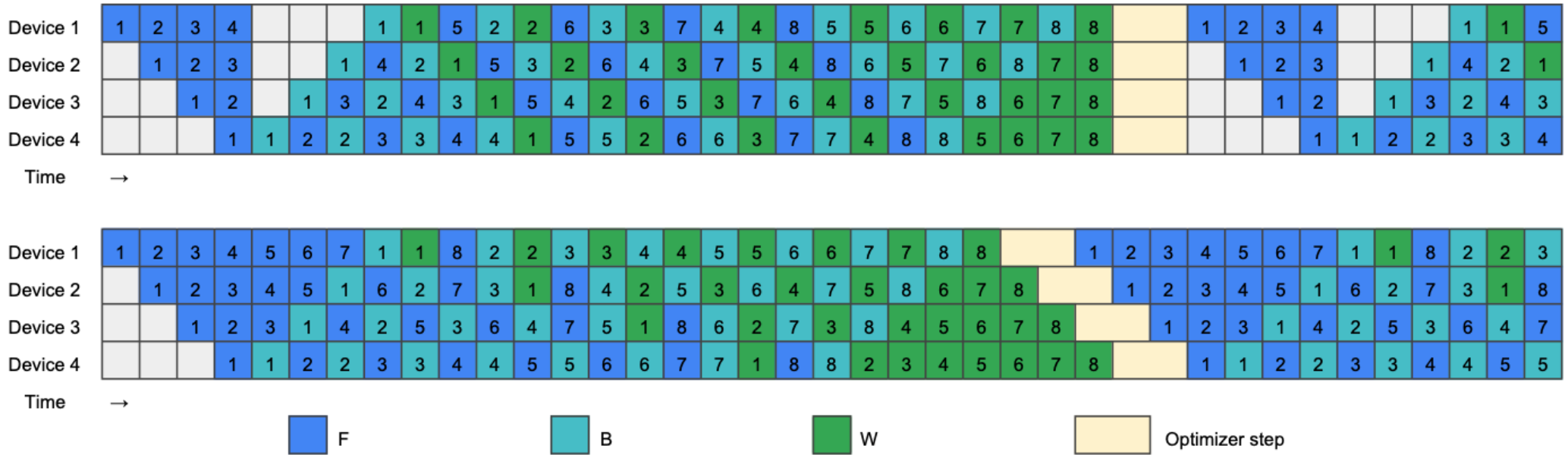


- Derivative over input
  - for backward propagation
- Derivative over weight
  - for updating model directly
- $\nabla_{\boldsymbol{W}} f(\boldsymbol{x}, \boldsymbol{W})^\top \frac{d\ell}{d\boldsymbol{y}}$ computed immediately after $\nabla_{\boldsymbol{x}} f(\boldsymbol{x}, \boldsymbol{W})^\top \frac{d\ell}{d\boldsymbol{y}}$

《Zero Bubble Pipeline Parallelism》

Pipeline Training

- Dividing BP into BP for input (urgent) and BP for weight (not urgent)

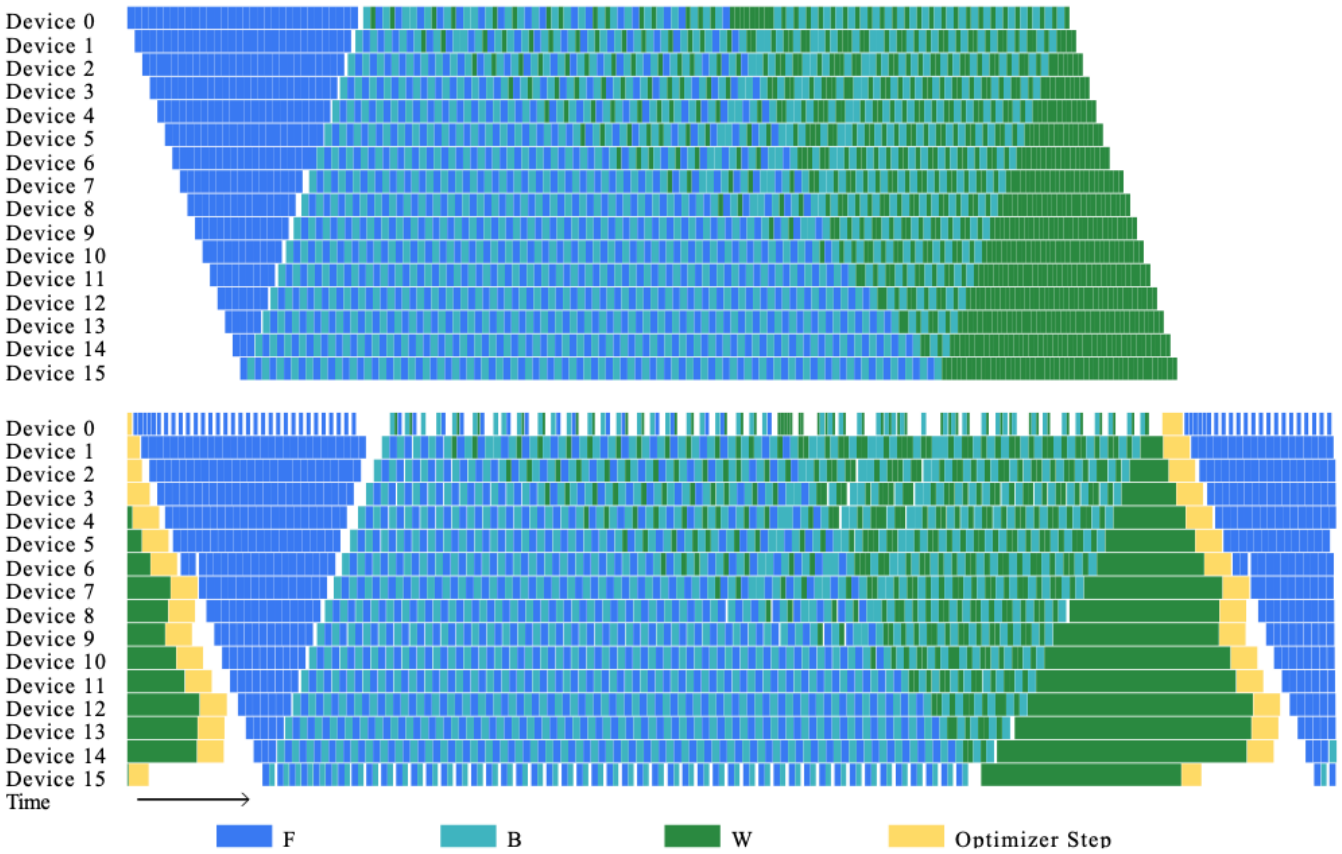


- Upper (ZB-H1): strictly follow 1F1B, and fill W to ensure all workers maintain the same number of in-flight microbatches.
- Lower (ZB-H2): theoretically zero bubble as # of microbatches is sufficiently large; large memory occupation over ZB-H1.

《Zero Bubble Pipeline Parallelism》

Pipeline Training

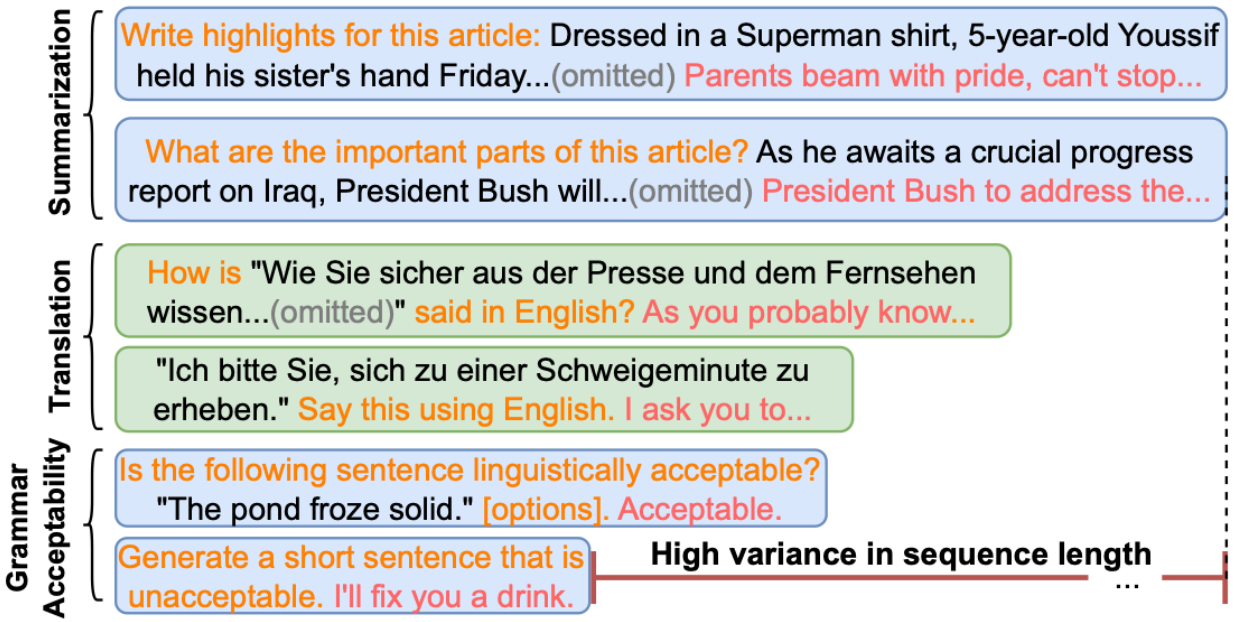
- An illustration of zero bubble with 16 stages: inserting W ops strategically

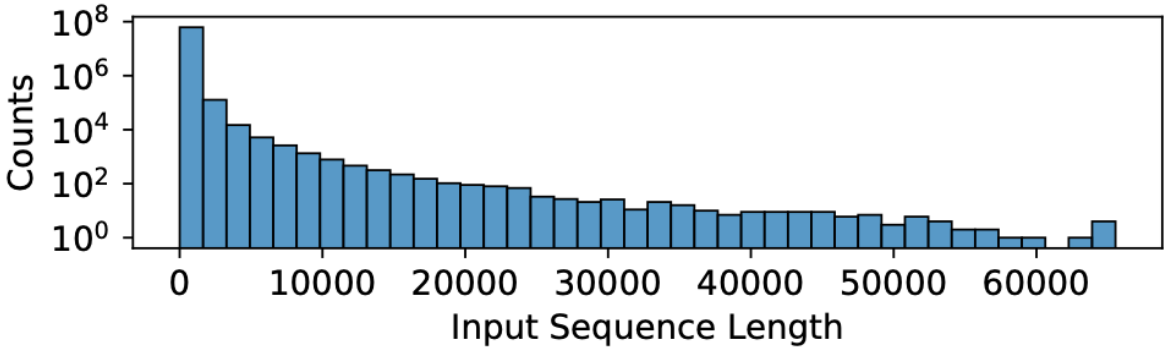


《Zero Bubble Pipeline Parallelism》

Pipeline Training

- Multi-task model pretraining
  - Input sequence lengths are non-uniform





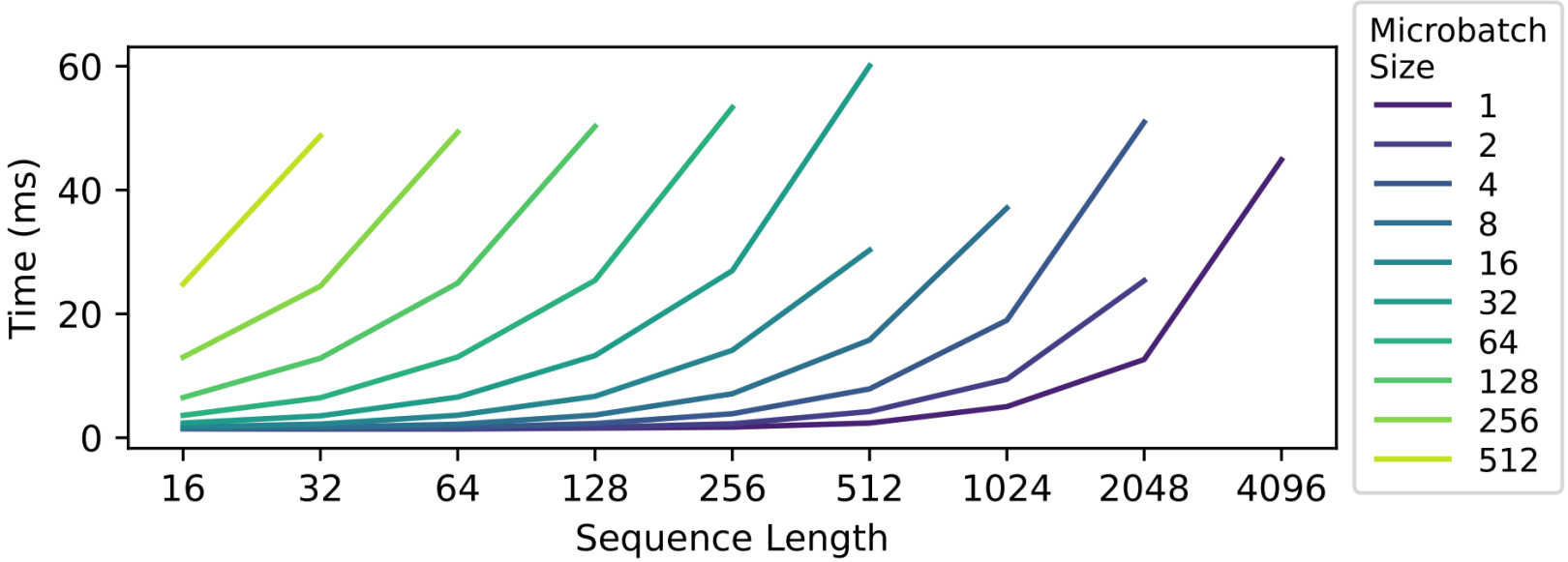
(left) Orange texts are instructions to the model. Inputs to process are colored black. Expected responses are in red.
(upper) The sequence length distribution in dataset (truncated at 65536).

《DynaPipe: Optimizing Multi-task Training through Dynamic Pipelines》

Pipeline Training

- Multi-task model pretraining
  - Execution time exhibits super-linear growth with sequence length

$$\text{FLOPS per Transformer Layer} = 24bsh^2 + 4b\boldsymbol{s^2}h$$



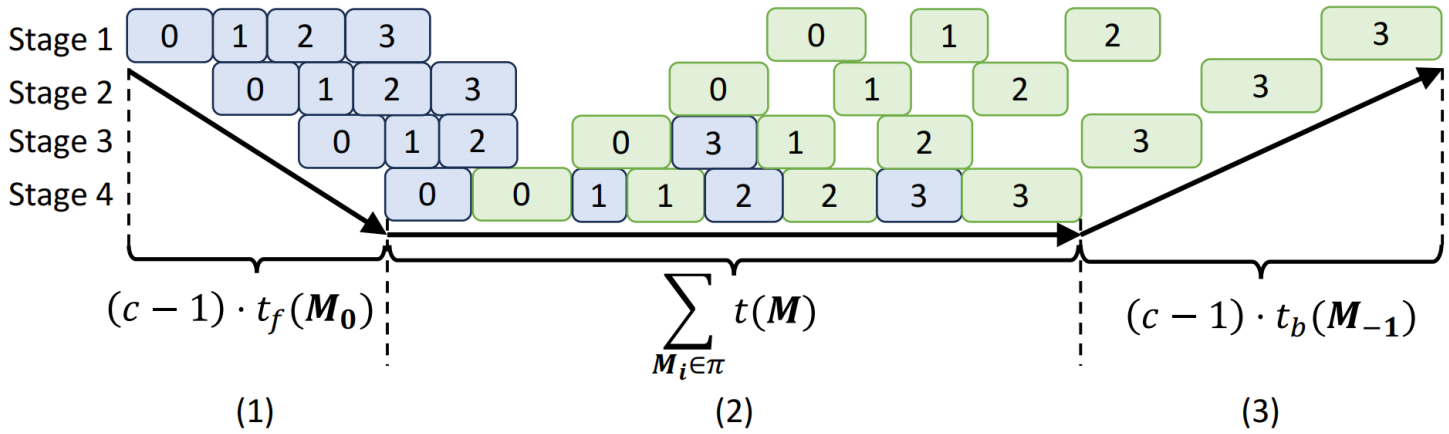
《DynaPipe: Optimizing Multi-task Training through Dynamic Pipelines》

Pipeline Training

- Multi-task model pretraining
  - Seek the best micro-batch assignment $\boldsymbol{\pi}$ to minimize iteration time

$$\min_{\boldsymbol{\pi}} \left\{ (c-1) \cdot \max\{t(\mathbf{M_i}) | \mathbf{M_i} \in \boldsymbol{\pi}\} + \sum_{\mathbf{M_i} \in \boldsymbol{\pi}} t(\mathbf{M_i}) \right\}$$

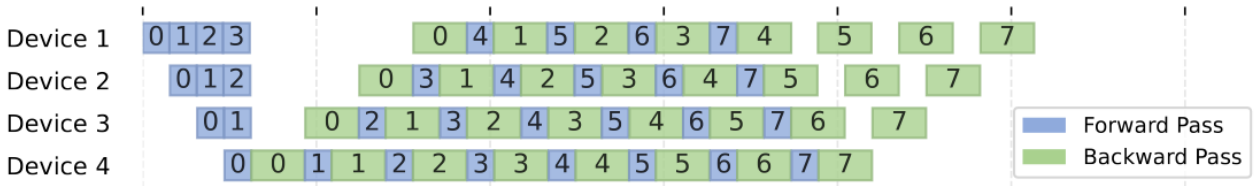
  - Construct a DP algorithm to optimally partition samples



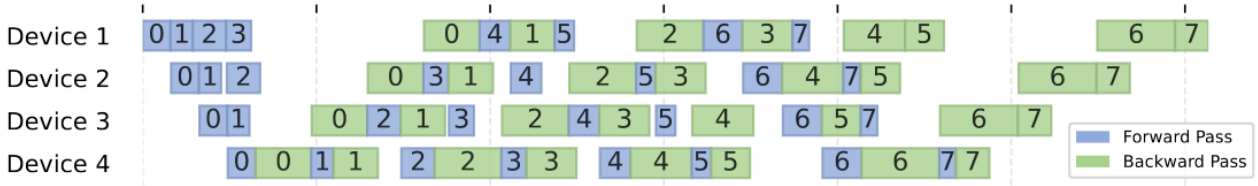
《DynaPipe: Optimizing Multi-task Training through Dynamic Pipelines》

Pipeline Training

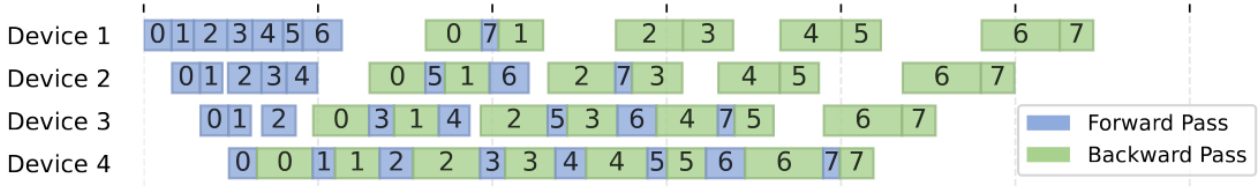
- Multi-task model pretraining
  - Packing and scheduling



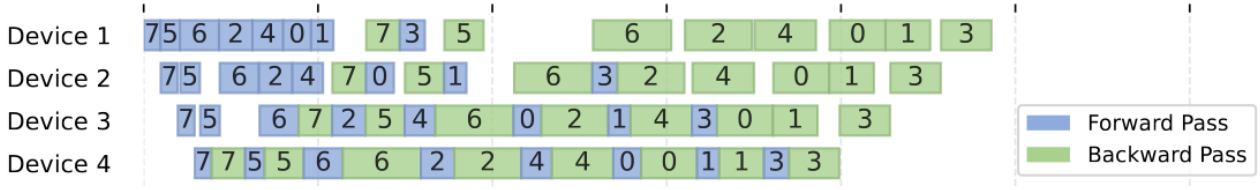
(a) 1F1B (uniform micro-batches)



(b) 1F1B (dynamic micro-batches)



(c) Adaptive Schedule



(d) Adaptive Schedule & Micro-batch Reordering

- Quite a few important studies on the similar problem
  - FlexPipe (ATC 2025), WLB-LLM (OSDI 2025), etc.

Thanks!